Hard link vs. soft link

Part A — Create the links
Create a directory named link_lab and navigate to it.
Create a file named security.txt containing the following text:
Linux File System Security Lab
Confidential Security Report

Create:
a hard link named security_hard.txt
a symbolic link named security_soft.txt

List all three files using a command that displays their inode numbers.

```
msis@master:~$ mkdir link_lab
msis@master:~$ cd link_lab
msis@master:~/link_lab$ nano security.txt
msis@master:~/link_lab$ ln security.txt security_hard.txt
msis@master:~/link_lab$ ln -s security.txt security_soft.txt
msis@master:~/link_lab$ ls -li
total 8
12192190 -rw-rw-r-- 2 msis msis 60 Sep  3 11:33 security_hard.txt
12192146 lrwxrwxrwx 1 msis msis 12 Sep  3 11:40 security_soft.txt -> security.txt
12192190 -rw-rw-r-- 2 msis msis 60 Sep  3 11:33 security.txt
```

Record the inode numbers of:
Security.txt
12192190
Security_hard.txt
12192146
Security_soft.txt
12192190

What difference do you observe?

Hard Link: Shares the **same inode**, increases the file's **link count**, and mirrors the exact file size.

Soft Link: Gets a **new inode**, starts with an **l** (link) attribute, and its size matches only the length of the filename text it points to.

Part B — Examine File Metadata

Use ls -l to examine the three files. Identify the file type of each file from the first character of the permission field.
Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```
msis@master:~/link_lab$ ls -l security.txt security_hard.txt security_soft.txt
-rw-rw-r-- 2 msis msis 60 Sep  3 11:33 security_hard.txt
lrwxrwxrwx 1 msis msis 12 Sep  3 11:40 security_soft.txt -> security.txt
-rw-rw-r-- 2 msis msis 60 Sep  3 11:33 security.txt
```

Use the stat command on all three files and record the following information:
Parameter Original Hard Link Soft Link
Inode number
File type
Size
Link count
Permissions

```
msis@master:~/link_lab$ stat security.txt
  File: security.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 259,7   Inode: 12192190    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    msis)   Gid: ( 1000/    msis)
Access: 2026-09-03 11:33:35.685938098 +0530
Modify: 2026-09-03 11:33:35.687422814 +0530
Change: 2026-09-03 11:39:56.631190464 +0530
 Birth: 2026-09-03 11:33:35.685938098 +0530
msis@master:~/link_lab$ stat security_hard.txt
  File: security_hard.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 259,7   Inode: 12192190    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    msis)   Gid: ( 1000/    msis)
Access: 2026-09-03 11:33:35.685938098 +0530
Modify: 2026-09-03 11:33:35.687422814 +0530
Change: 2026-09-03 11:39:56.631190464 +0530
 Birth: 2026-09-03 11:33:35.685938098 +0530
msis@master:~/link_lab$ stat security_soft.txt
  File: security_soft.txt -> security.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 259,7   Inode: 12192146    Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/    msis)   Gid: ( 1000/    msis)
Access: 2026-09-03 11:40:32.495677882 +0530
Modify: 2026-09-03 11:40:24.550569985 +0530
Change: 2026-09-03 11:40:24.550569985 +0530
 Birth: 2026-09-03 11:40:24.550569985 +0530
```

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

Identical Sizes: The original file and hard link share the exact same size because they point to the identical data blocks on the disk.

Path Text Only: The symbolic link size differs because it is an independent shortcut containing only the text string of the target file's path.

Character Count: Its size is exactly 12 bytes because the destination string "security.txt" contains exactly 12 characters.

Part C — Investigate readlink
Execute readlink on security_soft.txt. What information is displayed?
Execute readlink on security_hard.txt. What do you observe?
Based on your observations, explain why readlink works differently for hard links and symbolic links.
Use:
readlink -f security_soft.txt

```
msis@master:~/link_lab$ readlink security_soft.txt
security.txt
msis@master:~/link_lab$ readlink security_hard.txt
msis@master:~/link_lab$ echo $?
1
msis@master:~/link_lab$ readlink -f security_soft.txt
/home/msis/link_lab/security.txt
```

What does the -f option reveal?
The -f option reveals the canonical, absolute path of the target file by recursively following and resolving all symbolic links.

Part D — Content Modification
Append the following line to security.txt:
Unauthorized login detected

Display the contents using all three filenames.

```
msis@master:~/link_lab$ echo "Unauthorized login detected" >> security.txt
msis@master:~/link_lab$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
msis@master:~/link_lab$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
msis@master:~/link_lab$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
```

Is the newly added content visible through both links? Explain why.

Yes, the new content is instantly visible through both links.

- Hard Link Reason: It shares the identical disk inode as the original file, so they both read and write to the exact same physical storage blocks.
- Soft Link Reason: It acts as a shortcut pointing to the filename security.txt, meaning it automatically redirects the system to read whatever data currently exists at that path.

Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt.

```
msis@master:~/link_lab$ echo "Modified via hard link" >> security_hard.txt
msis@master:~/link_lab$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
Modified via hard link
msis@master:~/link_lab$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
Modified via hard link
```

What do you observe?

security.txt and security_soft.txt both show the new text added via the hard link.

The underlying data blocks are shared, meaning a change made through any filename immediately updates the content for all of them.

Part E — Rename Experiment

Rename:

security.txt

to:

incident.txt

Try accessing the content through security_hard.txt. Does it still work?

security_hard.txt: Yes, it still works. It successfully displays the full contents of the file.

Try accessing the content through security_soft.txt. Does it still work?

security_soft.txt: No, it does not work. It returns a No such file or directory error (creating a "broken link").

```
msis@master:~/link_lab$ mv security.txt incident.txt
msis@master:~/link_lab$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
Modified via hard link
msis@master:~/link_lab$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

```
msis@master:~/link_lab$ readlink security_soft.txt
security.txt
```

Explain why renaming the original file affects the symbolic link differently from the hard link.

Hard Link: Points directly to the inode number, so it doesn't care about the filename and continues to access the data under its new name.

Soft Link: Points strictly to the literal filename text, meaning it breaks instantly because the specific name it was targeting no longer exists.

Part F — Deletion Experiment

Recreate the links if necessary and then perform the following.

Delete the original file.

Check whether security_hard.txt can still be accessed.

Yes, it still works. It successfully displays the file's data.

Check whether security_soft.txt can still be accessed.

No, it does not work. It returns a No such file or directory error.

```
msis@master:~/link_lab$ rm security.txt
msis@master:~/link_lab$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Unauthorized login detected
Modified via hard link
msis@master:~/link_lab$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

Use ls -l and readlink to investigate the symbolic link after deleting its target.

```
msis@master:~/link_lab$ ls -l security_soft.txt
lrwxrwxrwx 1 msis msis 12 Sep  3 11:40 security_soft.txt -> security.txt
msis@master:~/link_lab$ readlink security_soft.txt
security.txt
```

What is a dangling/broken symbolic link?

A dangling or broken symbolic link is a shortcut file whose stored text path points to a filename that has been deleted or moved, leaving it pointing to a nonexistent target.

Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

No. Linux only deletes underlying data when an inode's link count drops to zero. Because the hard link shares the same inode as the original file, deleting the original filename merely decreases the link count from 2 to 1, leaving the inode active and the data intact.

Part G — Directory and Filesystem Behaviour

Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

Hard Link Directory Result: Failed. Linux blocks hard-linking directories to prevent infinite loops/cycles in the filesystem structure.

Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

Soft Link Directory Result: Successful. Symbolic links are path shortcuts and safely target any directory structure.

```
msis@master:~/link_lab$ mkdir evidence
msis@master:~/link_lab$ ln evidence evidence_hard.txt
ln: evidence: hard link not allowed for directory
msis@master:~/link_lab$ ln -s evidence evidence_link
```

Based on your experiment, compare hard and symbolic links with respect to directory linking.

Directory Linking Comparison: Hard links cannot target directories; symbolic links can target directories.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

a hard link across filesystems

a symbolic link across filesystems

```
msis@master:~/link_lab$ ln /etc/hostname /tmp/hostname_hard 2>/dev/null || ln security_hard.txt /run/test_hard
ln: failed to create hard link '/run/test_hard' => 'security_hard.txt': Invalid cross-device link
msis@master:~/link_lab$ ln -s /etc/hostname /tmp/hostname_soft
```

Cross-Filesystem Link Comparison: Hard links cannot cross filesystem boundaries (they rely on filesystem-specific inode indexes); symbolic links can cross filesystems because they store plain path text strings

Final Comparison

Based only on your experimental observations, complete the following table:

Parameter Hard Link Symbolic Link

Inode number

File type

Link count

File size

Stores target pathname?

readlink output

Effect of modifying target

Effect of renaming target

Effect of deleting target

Can become a dangling or broken link?

Directory linking

Cross-filesystem linking

Final Lab Comparison Table

Parameter	Hard Link	Symbolic Link
Inode number	**Identical** to the target	**Unique** independent inode
File type	Regular file (-)	Symbolic Link (l)
Link count	**Increases** by 1	Remains **1**
File size	Identical to target size	Equal to character length of path string
Stores target pathname?	**No**	**Yes**
readlink output	Empty string + Error code	Prints the target path string
Effect of modifying target	Changes are **instantly visible**	Changes are **instantly visible**

Effect of renaming target	Continues working normally	**Breaks** (link becomes broken)
Effect of deleting target	Continues working normally	**Breaks** (becomes a dangling link)
Can become dangling/broken?	**No**	**Yes**
Directory linking	**Not allowed**	**Allowed**
Cross-filesystem linking	**Not allowed**	**Allowed**

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

172.16.0.5 guest 2

10.0.0.20 admin 8

Format:

IP-address username failed attempts

1.Sort according to failed attempts

2.Sort a list of suspicious IP addresses alphabetically.

3.Display only unique suspicious IP addresses using sort.

4.Sort a list of detected port numbers numerically.

5.Display port numbers in descending numerical order.

6.Sort usernames without considering uppercase/lowercase differences.

7.From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

8.Given IP,username,failed_attempts, sort records based on the number of failed attempts.

9.Sort a CSV vulnerability report based on severity score.

10.Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

11.Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #1
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ sort -k3 -n alerts.txt
IP-address     username failed attempts
172.16.0.5        guest   2
192.168.1.10      admin   5
10.0.0.20         admin   8
10.0.0.8           root   12
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #2
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $1}' alerts.txt | sort
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
IP-address
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #3
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $1}' alerts.txt | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
IP-address
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #4
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $3}' alerts.txt | sort -n
failed
2
5
8
12
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #5
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $3}' alerts.txt | sort -nr
12
8
5
2
failed
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #6
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $2}' alerts.txt | sort -f
admin
admin
guest
root
username
```

```
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #7
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ awk '{print $1}' alerts.txt | sort | uniq -c | sort -nr
      1 IP-address
      1 192.168.1.10
      1 172.16.0.5
      1 10.0.0.8
      1 10.0.0.20
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #8
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ sort -k3 -n alerts.txt
IP-address     username failed attempts
172.16.0.5        guest   2
192.168.1.10      admin   5
10.0.0.20         admin   8
10.0.0.8           root   12
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #9
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ sort -k3 -n alerts.txt
IP-address     username failed attempts
172.16.0.5        guest   2
192.168.1.10      admin   5
10.0.0.20         admin   8
10.0.0.8           root   12
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #10
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' alerts.txt | sort | uniq
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ #11
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Assign4$ sort -k3 -nr alerts.txt | awk '{print $1}' | head -n 5
10.0.0.8
10.0.0.20
192.168.1.10
172.16.0.5
IP-address
```